

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: Новые языки программирования

Студент гр. 4388

Максимова М.А.

Студент гр. 4388

Ходыко Е.А.

Студент гр. 4388

Тарабанов А.И.

Руководитель

Шестопалов Р.П.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Максимова М.А. Группа: 4388

Студент: Ходыко Е.А. Группа: 4388

Студент: Тарабанов А.И. Группа: 4388

Тема практики: Новые языки программирования

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Java с графическим интерфейсом.

Алгоритм: Прима.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	Максимова М.А.
Студент	Ходыко Е.А.
Студент	Тарабанов А.И.
Руководитель	Шестопалов Р.П.

АННОТАЦИЯ

В отчёте представлена разработка интерактивного приложения для визуализации алгоритма Прима на языке Java с использованием библиотеки Swing. Программа позволяет пользователю создавать и редактировать взвешенный граф, пошагово наблюдать выполнение алгоритма, а также импортировать/экспортировать данные. Описаны архитектура, функциональные требования, процесс реализации и тестирования. Приложение предназначено для образовательных целей и демонстрации работы алгоритма построения минимального остовного дерева.

SUMMARY

The report presents the development of an interactive application for visualizing Prim's algorithm in Java using the Swing library. The program allows the user to create and edit a weighted graph, observe the algorithm step by step, and import/export data. The architecture, functional requirements, implementation process, and testing are described. The application is intended for educational purposes and for demonstrating the minimum spanning tree algorithm.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	7
1.1 Предметная область	7
1.2 Задачи практики	7
2 РЕЗУЛЬТАТЫ РАБОТЫ	8
2.1. Вводное задание:	8
2.2. Составление спецификации программы и плана разработки	8
1. Назначение	8
2.1. Ввод данных.....	9
2.2. Визуализация графа и алгоритма.....	9
2.3. Вывод результатов.....	10
2.4. Дополнительные возможности	10
3. Пользовательский интерфейс(референс):	10
4. Принцип работы (визуализация алгоритма):	12
5. Формат входных и выходных данных.....	13
5.1. Входной файл (импорт графа)	13
5.2. Выходной файл (экспорт результата).....	13
ПЛАН РЕАЛИЗАЦИИ	16
2.2.1 Исходные требования к программе.....	19
2.2.2 План разработки и распределение ролей в бригаде	19
2.2.3 Уточнение требований после сдачи 1-ой версии	20
2.2.4 Уточнение требований после сдачи 2-ой версии	20
2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом.....	21
2.3.1. Структуры данных	22
2.3.2. Основные методы.....	22
2.3.3. Тестирование	24
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	34
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	35

ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38
ПРИЛОЖЕНИЕ А	39

ВВЕДЕНИЕ

Предметная область – визуализация алгоритмов на графах. Такие инструменты широко используются в образовательном процессе при изучении дискретной математики, теории алгоритмов и программирования. Наглядное представление шагов алгоритма помогает глубже понять его логику и особенности работы.

Цель практики – разработать интерактивное приложение для визуализации алгоритма Прима, позволяющее пользователю создавать граф, запускать алгоритм пошагово или в автоматическом режиме, а также анализировать результаты.

Задачи практики:

- Изучить необходимые технологии (язык Java, библиотеку Swing, работу с графическим интерфейсом и событиями мыши).
- Составить спецификацию программы и план разработки.
- Реализовать приложение в команде, применяя итеративный подход, с последующим тестированием и документированием.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Алгоритм Прима – один из классических алгоритмов для нахождения минимального остовного дерева (MST) во взвешенном неориентированном графе. Теоретические основы алгоритма описаны в классическом учебнике по алгоритмам [1]. Он широко применяется в задачах сетевого проектирования, кластерного анализа и других областях. Визуализация алгоритма позволяет студентам и исследователям наблюдать за процессом выбора рёбер, что способствует лучшему усвоению материала. Разработанное приложение может быть использовано на практических занятиях по курсам «Дискретная математика» и «Алгоритмы и структуры данных».

1.2 Задачи практики

В рамках практики были поставлены следующие задачи:

1. **Вводное задание:** изучение языка Java, библиотеки Swing, принципов событийно-ориентированного программирования и работы с графикой.
2. **Составление спецификации программы и плана разработки:** определение функциональных требований, описание архитектуры, форматов входных/выходных данных, распределение ролей в бригаде.
3. **Командная итеративная разработка приложения:** реализация моделей данных, алгоритма Прима, графического интерфейса, функций импорта/экспорта, генерации случайного графа, а также тестирование и отладка.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание:

Все студенты группы в качестве вводного задания выбрали прохождение онлайн-курса Stepik “[Java. Базовый курс](#)”. Дополнительно для углублённого изучения языка использовалась литература [2]. Ходыко Е.А. и Максимова М.А. выполнили и защитили требуемый минимум курса:

- Введение в Java
- Базовый синтаксис Java
- Объекты, классы и пакеты в Java
- Знакомство с исключениями
- Обработка исключений. Try-catch
- Доступ к файловой системе

Тарабанов А.И. выполнил и защитил(помимо указанных выше) также пункты курса:

- Обработка ошибок, исключения, отладка
- Ввод-вывод, доступ к файловой системе
- Generics. Collections. Streams

2.2. Составление спецификации программы и плана разработки

1. Назначение

Программа предназначена для демонстрации работы алгоритма Прима построения минимального остовного дерева во взвешенном неориентированном графе. Она позволяет пользователю:

- вводить граф в интерактивном режиме
- визуально наблюдать пошаговое выполнение алгоритма
- получать итоговое минимальное остовное дерево и его вес
- экспортировать результаты

Программа ориентирована на образовательные цели, а также может использоваться для проверки правильности работы алгоритма на пользовательских данных.

2. Функциональные требования

2.1. Ввод данных

Программа поддерживает два равноправных способа задания графа:

1. Интерактивный ввод (в приложении). Пользователь создаёт граф непосредственно на холсте с помощью мыши:

- Вершина добавляется, щелчком левой кнопки мыши по свободному месту холста.
- Ребро создаётся путём выделения двух вершин (последовательными щелчками с зажатой клавишей Ctrl или с помощью перетаскивания от одной вершины к другой). После выбора вершин появляется диалоговое окно для ввода веса ребра (положительное целое или вещественное число).
- Вершины можно перемещать по холсту перетаскиванием.
- Вершину или ребро можно удалить через контекстное меню (правая кнопка мыши) или клавишей Delete при выделении.

2. Импорт из файла. Файл должен быть в текстовом формате (подробности в разделе 5).

В обоих случаях граф должен быть связным и взвешенным (веса > 0). При попытке запустить алгоритм на несвязном графе программа выводит предупреждение и не начинает выполнение. Также проверяется отсутствие петель и кратных рёбер (при необходимости).

2.2. Визуализация графа и алгоритма

Граф будет отображаться в виде вершин (кругов) и рёбер (линий) с подписями весов.

На каждом шаге алгоритма могут подсвечиваются:

- текущая вершина, добавляемая в остов

- ребро, добавляемое в остов (выделяется цветом)
- множество уже построенных вершин и рёбер
- кандидаты на добавление (рёбра, пересекающие разрез) с их весами

Панель управления содержит кнопки:

- «Следующий шаг» – переход к следующей итерации;
- «Автозапуск» – непрерывное выполнение с регулируемой скоростью (регуляция при помощи ползунка);
- «Сброс» – возврат к исходному графу;
- «Завершить» – переход к финальному состоянию.

Визуализация поддерживает масштабирование и перемещение графа (zoom).

2.3. Вывод результатов

После завершения работы алгоритма программа отображает:

- минимальное остовное дерево (подсвеченные рёбра);
- общий вес дерева;
- список рёбер, входящих в MST, с указанием порядка их добавления.

А также возможность сохранить результаты в текстовый файл (формат – см. раздел 5.1).

2.4. Дополнительные возможности

- Генерация случайного графа по заданному числу вершин и плотности рёбер.
- Отображение таблицы состояний (для каждой вершины – расстояние до остова и родительская вершина).

3. Пользовательский интерфейс(референс)(рис. 1):

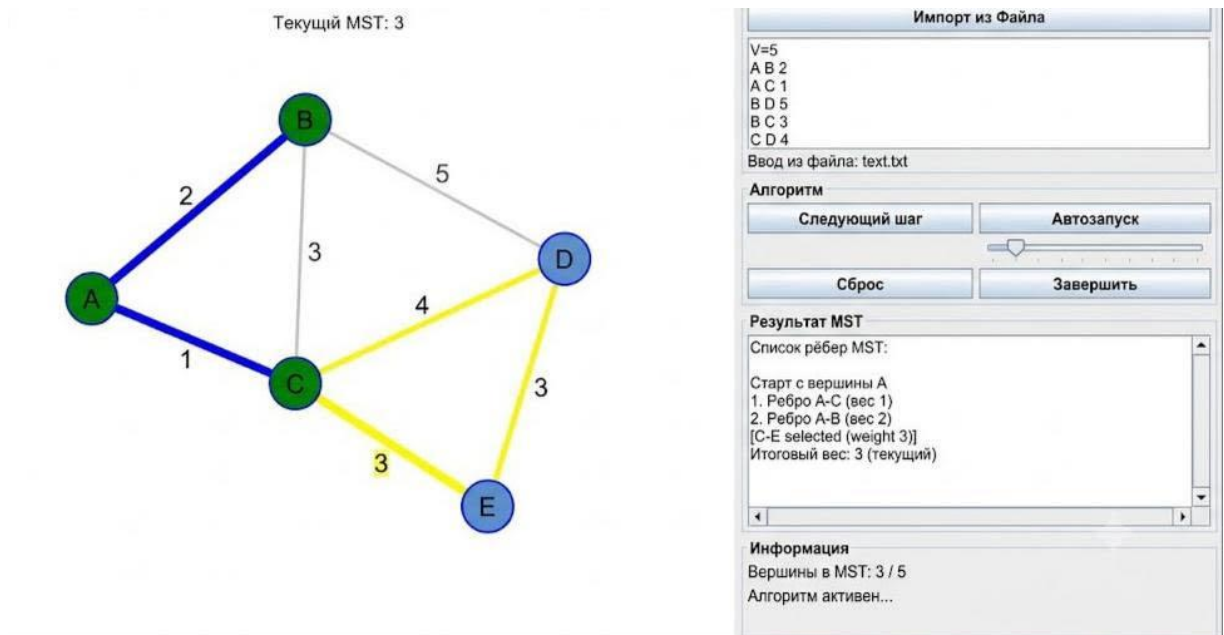


Рисунок 1 – референс для пользовательского интерфейса для визуализации работы алгоритма Прима

Планируется следующее взаимодействие с холстом с помощью мыши:

- **Клик по пустому месту** – создание новой вершины (в режиме редактирования графа).
- **Последовательный клик по двум вершинам с зажатой клавишей Ctrl** (или перетаскивание от одной вершины к другой) – создание ребра; после этого открывается диалог ввода веса.
- **Перетаскивание вершины** – изменение её положения.
- **Щелчок правой кнопкой по вершине/ребру** – контекстное меню с опциями «Удалить», «Изменить вес».
- **Прокрутка колёсиком мыши** – масштабирование холста.
- **Перетаскивание фона (зажатая средняя кнопка / левая кнопка на свободном месте)** – панорамирование.

4. Принцип работы (визуализация алгоритма):

Программа реализует алгоритм Прима, начиная с произвольно выбранной стартовой вершины. На каждом шаге:

1. Выделяется множество уже построенных вершин (закрашиваются зелёным)
2. Выделяются рёбра, соединяющие построенные и непостроенные вершины (кандидаты) – показываются жёлтым цветом
3. Среди кандидатов выбирается ребро с минимальным весом – оно подсвечивается мигающим цветом (красный)
4. Добавляется новая вершина и ребро в остов – они окрашиваются в синий
5. На панели информации обновляется текущий вес остова и номер шага
6. Если остались неподключённые вершины, алгоритм переходит к следующему шагу

После завершения все рёбра остовного дерева отображаются жирной синей линией, остальные рёбра становятся полупрозрачными.

Псевдокод алгоритма см. в разделе 6.

5. Формат входных и выходных данных

5.1. Входной файл (импорт графа)

Текстовый файл с расширением .txt. Структура:

1. Первая строка – одно целое число N – количество вершин.
2. Вторая строка – одно целое число M – количество рёбер.
3. Следующие M строк описывают рёбра: $u\ v\ w$, где u, v – номера вершин (начиная с 0 или 1, указывается), w – вес ребра (целое или вещественное > 0).

Пример:

```
4
5
0 1 2.5
0 2 1.0
1 2 3.0
1 3 4.0
2 3 0.5
```

5.2. Выходной файл (экспорт результата)

При сохранении результатов создаётся текстовый файл, содержащий:

- Общий вес минимального остовного дерева.
- Список рёбер дерева в порядке их добавления с указанием веса.

Пример выходного файла:

Minimal Spanning Tree Weight: 4.0

Edges added (in order):

- 1: (0, 2) weight 1.0
- 2: (2, 3) weight 0.5
- 3: (0, 1) weight 2.5

Формат совместим с любым текстовым редактором для просмотра и печати.

6. Алгоритм Прима (псевдокод)

Вход: связный взвешенный граф $G(V, E)$

Выход: множество рёбер MST, общий вес

visited = пустое множество

mst_edges = []

total_weight = 0

Выбрать произвольную стартовую вершину start

visited.add(start)

Пока visited $\neq V$:

 candidates = все рёбра, у которых один конец в visited,

 а другой – нет

 найти ребро $e = (u, v)$ среди candidates с минимальным весом w

 visited.add(v)

 mst_edges.append(e)

 total_weight += w

 Визуализировать шаг (обновить холст, пояснения)

Вернуть (mst_edges, total_weight)

ПЛАН РЕАЛИЗАЦИИ

Этап 1. Проектирование архитектуры и выбор библиотек

Структура проекта:

- **model** – модели данных (вершина, ребро, граф, состояние алгоритма).
- **algorithm** – реализация алгоритма Прима с пошаговым состоянием. Возможно заберем оттуда сортировку.
- **ui** – компоненты интерфейса (холст, панель управления, контекстное меню).
- **io** – импорт/экспорт файлов.
- **util** – вспомогательные утилиты (валидация, генерация случайного графа).

Этап 2. Модели данных (**model**)

Реализовать неизменяемые или контролируемо изменяемые классы:

- **Vertex**: id, координаты (x, y), цвет состояния (не в осто́ве / в осто́ве и т.д.).
- **Edge**: ребро, вес, цвет/стиль линии, флаг «в MST».
- **Graph**: коллекции вершин и рёбер, методы проверки связности, отсутствия петель/кратных рёбер.
- **PrimState**: текущее состояние алгоритма (множество посещённых вершин, список кандидатов, текущее минимальное ребро, шаг, накопленный вес). Это нужно для пошаговой визуализации.

Этап 3. Реализация алгоритма Прима

- Реализовать алгоритм как итератор состояний: каждый вызов `nextStep()` возвращает новое состояние `PrimState`, соответствующее одной итерации. Это позволяет легко управлять автозапуском.

- Использовать PriorityQueue для эффективного поиска минимального ребра среди кандидатов.
- Добавить проверку связности графа перед запуском (например, через DFS/BFS, посмотрим). Если граф несвязный – вернуть ошибку/предупреждение.
- Поддерживать веса как double.

Этап 4. Пользовательский интерфейс и холст (ui)

Попробуем реализовать через JavaFX. Можно присвоить разным клавишам различные назначения, например, зум на колесико, схватить и тащить вершину на правую кнопку мыши. Считается, что лучше всего экспериментировать и в процессе выявить лучшие решения и применить их в работе, чем заранее придумывать как все должно выглядеть, опираясь только на теоретическую основу. Референс есть в разделе 3, дублируем основные идеи взаимодействия с холстом:

- **Клик по пустому месту** – создание новой вершины (в режиме редактирования графа).
- **Последовательный клик по двум вершинам с зажатой клавишей Ctrl** – создание ребра; после этого открывается диалог ввода веса.
- **Перетаскивание вершины** – изменение её положения.
- **Щелчок правой кнопкой по вершине/ребру** – контекстное меню с опциями «Удалить», «Изменить вес».
- **Прокрутка колёсиком мыши** – масштабирование холста.
- **Перетаскивание фона (зажатая средняя кнопка / левая кнопка на свободном месте)** – панорамирование.

Этап 5. Визуализация шагов алгоритма

При каждом шаге обновлять цвета элементов согласно спецификации:

- Посещённые вершины – зелёный.
- Кандидаты (рёбра через разрез) – жёлтый.
- Минимальное ребро на шаге – мигающий красный (анимировать)
- Добавленные вершины/рёбра – синий.
- В финале: рёбра MST – жирные синие, остальные – полупрозрачные.

Обновлять текстовую панель: текущий вес, номер шага

Этап 6. Ввод/вывод и дополнительные функции

- **Импорт графа** из .txt по формату из спецификации (разбор строк, валидация весов > 0 , проверка на петли/кратные рёбра).
- **Экспорт результата** в текстовый файл (вес MST, список рёбер в порядке добавления).
- **Генерация случайного графа**: по числу вершин и плотности (вероятность ребра между парами), веса как случайные положительные double.

Этап 7. Тестирование и отладка

- Проверить корректность MST на нескольких ручных примерах (сравнить с ручной проверкой или другим алгоритмом, например, Краскала).
- Протестировать крайние случаи: 1 вершина, 2 вершины, несвязный граф, граф с одинаковыми весами.
- Убедиться, что визуализация корректно отражает каждый шаг (цвета, порядок рёбер).
- Проверить обработку ошибок: некорректный формат файла, отрицательные веса, дубли рёбер.

Этап 8. Сборка и публикация

- Настроить сборку
- Собрать JavaFX-приложение (например, как JAR)
- Составить понятный README с инструкцией по запуску и форматами файлов для удобства пользователей.
- Опционально: добавить возможность пользователей отправлять сведения об ошибках или просто выкладывать свои интересные примеры деревьев.

2.2.1 Исходные требования к программе

Ввод данных:

- Интерактивное создание графа на холсте: добавление вершин щелчком мыши, создание рёбер с указанием веса, перемещение и удаление элементов.
- Загрузка графа из текстового файла формата: N (число вершин), M (число рёбер), затем M строк «u v w».

Визуализация алгоритма:

- Пошаговый режим с отображением текущего состояния: посещённые вершины (зелёным), рёбра-кандидаты (жёлтым), выбираемое ребро (мигает красным), добавленные рёбра (синим).
- Автоматический режим с регулируемой скоростью.
- Отображение текущего веса остова и списка добавленных рёбер.

Вывод результатов:

- Итоговое остовное дерево с выделением рёбер.
- Общий вес дерева.
- Возможность сохранения результата в текстовый файл.

Дополнительно:

- Генерация случайного графа по заданному числу вершин и плотности.
- Масштабирование и панорамирование холста.

2.2.2 План разработки и распределение ролей в бригаде

Разработка разбита на этапы:

1. **Проектирование архитектуры** (все участники) – определение структуры классов, интерфейсов, выбор библиотек.
2. **Реализация модели данных** (Ходыко Е.) – классы Vertex, Edge, Graph, состояние алгоритма.
3. **Реализация алгоритма Прима** (Максимова М.) – пошаговый итератор, использование PriorityQueue.
4. **Графический интерфейс и холст** (Тарабанов А.) – панель рисования, элементы управления, обработка событий мыши.
5. **Ввод/вывод и дополнительные функции** (совместно) – импорт/экспорт файлов, генерация случайного графа.
6. **Тестирование и отладка** (все участники).
7. **Сборка и документирование** (Тарабанов А.).

Промежуточные версии:

- **Прототип** – базовая отрисовка графа, добавление вершин/рёбер.(сразу была предоставлена 1-ая версия)
- **Версия 1** – реализация алгоритма с пошаговой визуализацией.
- **Версия 2** – автозапуск, импорт/экспорт, случайная генерация.
- **Финальная версия** – полировка интерфейса, исправление ошибок, подготовка отчёта.

2.2.3 Уточнение требований после сдачи 1-ой версии

Добавить взаимодействия с мышью и кнопками(интерактивное взаимодействие), добавить возможность сохранения в файл результата работы программы.

2.2.4 Уточнение требований после сдачи 2-ой версии

Добавить возможность для пользователя увидеть основную информацию о разработчиках при нажатии соответствующей кнопки в приложении.

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

Разработка велась в соответствии с планом, все этапы были выполнены в срок. Процесс организован с использованием системы контроля версий Git (репозиторий на GitHub). Исходный код проекта доступен по ссылке [3]. Каждый участник вносил изменения в свою часть кода, проводились код-ревью и совместное тестирование.

Проект организован по принципу модульности. Все классы разбиты по пакетам:

- `main` – содержит точку входа `PrimAlgorithmVisualizer.java` (главный класс приложения, который наследует `JFrame` и управляет режимами, моделью и контроллерами).
- `model` – содержит классы данных:
 - `Vertex.java` – вершина графа (поля: `id`, координаты, метка).
 - `Edge.java` – ребро графа (поля: `id`, номера вершин-концов, вес, метод `connects` для проверки инцидентности).
- `ui` – содержит компоненты пользовательского интерфейса:
 - `GraphPanel.java` – панель для рисования графа и визуализации алгоритма (обрабатывает события мыши, отрисовку, зум/панорамирование).
 - Остальные элементы управления (кнопки, панели) собраны непосредственно в главном классе `PrimAlgorithmVisualizer`, однако в перспективе их также можно вынести в отдельные классы.
 - В этом же пакете лежит исполняемый JAR-файл `PrimVisualizer.jar` и служебные файлы `.DS_Store` (можно игнорировать).

Такое разделение позволяет упростить поддержку и тестирование: логика данных отделена от визуализации, а управление вынесено в основной класс.

2.3.1. Структуры данных

- **Vertex:** хранит идентификатор, координаты, метку (буквенное обозначение).
- **Edge:** содержит идентификатор, номера вершин, вес.
- **Граф** представлен списками вершин и рёбер в основном классе `PrimAlgorithmVisualizer`. Методы для проверки связности, поиска рёбер между вершинами и валидации.
- **Состояние алгоритма:** множество посещённых вершин (`inTree`), список рёбер MST, общий вес, флаг завершения.

2.3.2. Основные методы

- **Редактирование**

графа: `addVertex()`, `removeVertex()`, `removeEdge()`, `tryCreateEdge()`.

Алгоритм Прима:

- `resetAlgorithmState()` – инициализация с произвольной стартовой вершиной.
- `getCandidateEdges()` – возвращает рёбра, пересекающие разрез (один конец в осто́ве, другой вне).
- `nextStep(boolean animate)` – выполняет один шаг: находит минимальное ребро-кандидат, анимирует его (мигание) и добавляет в остов.
- `commitStep(Edge e)` – фиксирует добавление ребра, обновляет множество и вес.
- `stepBack()` – откатывает последний шаг.
- **Визуализация:** метод `paintComponent()` класса `GraphPanel` отвечает за отрисовку всех элементов с учётом текущего режима и состояния алгоритма. Используются цвета и толщины линий в соответствии со спецификацией.
- **Управление видом:** масштабирование (`zoomAt`), панорамирование, сброс вида.

- **Ввод/вывод:** методы loadGraphFromFile() (разбор текстового файла) и saveResultsToFile() (экспорт результата).

В дополнение к уже приведённому описанию уточним, какие методы относятся к каким классам:

Модель (model)

- Vertex – конструктор, геттеры.
- Edge – конструктор, метод connects(int a, int b).

Главный класс (PrimAlgorithmVisualizer)

- addVertex, removeVertex, removeEdge, tryCreateEdge – редактирование графа.
- resetAlgorithmState – инициализация алгоритма.
- getCandidateEdges – вычисление рёбер-кандидатов.
- nextStep и commitStep – пошаговое выполнение с анимацией.
- stepBack – откат последнего шага.
- loadGraphFromFile и saveResultsToFile – импорт/экспорт.
- toggleMode – переключение между режимами.

Панель рисования (GraphPanel)

- paintComponent – вся отрисовка графа, рёбер, вершин, подписей, легенды.
- Обработчики мыши (mousePressed, mouseDragged, mouseReleased) – реализуют интерактивность (добавление, перемещение, создание рёбер, контекстное меню, панорамирование, масштабирование).
- Вспомогательные методы преобразования координат (toWorldX, toScreenX и т.д.) и поиска объектов под курсором.

Алгоритм – непосредственно заложен в методах nextStep и commitStep, где используется жадный выбор минимального ребра-кандидата.

2.3.3. Тестирование

Тестирование разработанного приложения проводилось вручную на всех ключевых функциональных блоках. Ввиду графической природы интерфейса автоматизированное тестирование не применялось, однако каждый этап был задокументирован с помощью скриншотов, которые приведены ниже.

2.3.3.1. План тестирования

Перед началом тестирования был составлен план, охватывающий следующие направления:

1. **Корректность работы алгоритма Прима** – проверка на графах с известным минимальным остовным деревом (эталонные результаты получены вручную или с помощью альтернативного алгоритма Краскала).
2. **Функциональность редактирования графа** – добавление, удаление, перемещение вершин и рёбер, изменение весов, проверка валидации ввода.
3. **Импорт и экспорт данных** – загрузка графа из текстового файла, сохранение результатов в файл, обработка ошибочных форматов.
4. **Управление визуализацией** – пошаговый режим, автозапуск с регулировкой скорости, сброс, завершение, шаг назад.
5. **Интерактивные элементы интерфейса** – масштабирование, панорамирование, контекстные меню, клавиатурные сокращения.

2.3.3.2. Тестирование графического интерфейса

Все элементы управления и взаимодействие с холстом были проверены на соответствие спецификации. Основные сценарии:

- **Создание и перемещение вершин:** при щелчке по пустому месту холста появляется новая вершина с уникальной буквенной меткой (рис. 2). Перетаскивание вершины меняет её положение, при этом инцидентные рёбра перерисовываются.

- **Создание рёбер:** с помощью Ctrl+клик по двум вершинам или Shift+перетаскивание от вершины к вершине открывается диалог ввода веса (рис. 3, рис. 4). Вес должен быть положительным числом; при вводе некорректного значения выводится предупреждение.
- **Редактирование и удаление:** через контекстное меню (ПКМ) можно удалить вершину или ребро, а также изменить вес ребра (рис. 5). Выделенный объект можно удалить клавишей Delete/Backspace.
- **Панорамирование и масштабирование:** Alt+перетаскивание фона перемещает вид, а колесо мыши плавно изменяет масштаб (рис. 6). Индикатор масштаба в верхней панели обновляется.
- **Загрузка графа из файла:** при выборе файла .txt программа автоматически определяет индексацию (с 0 или с 1) и выводит успешно загруженный граф (рис. 7). В случае ошибки формата показывается понятное предупреждение.

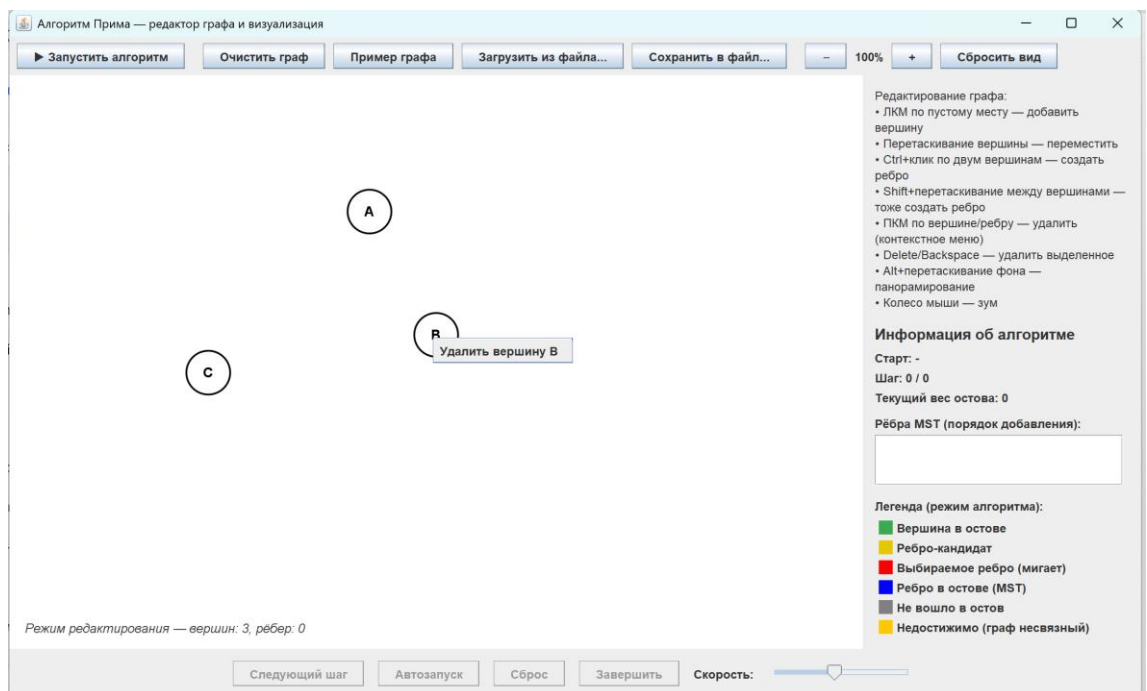


Рисунок 2 — Добавление вершины в режиме редактирования

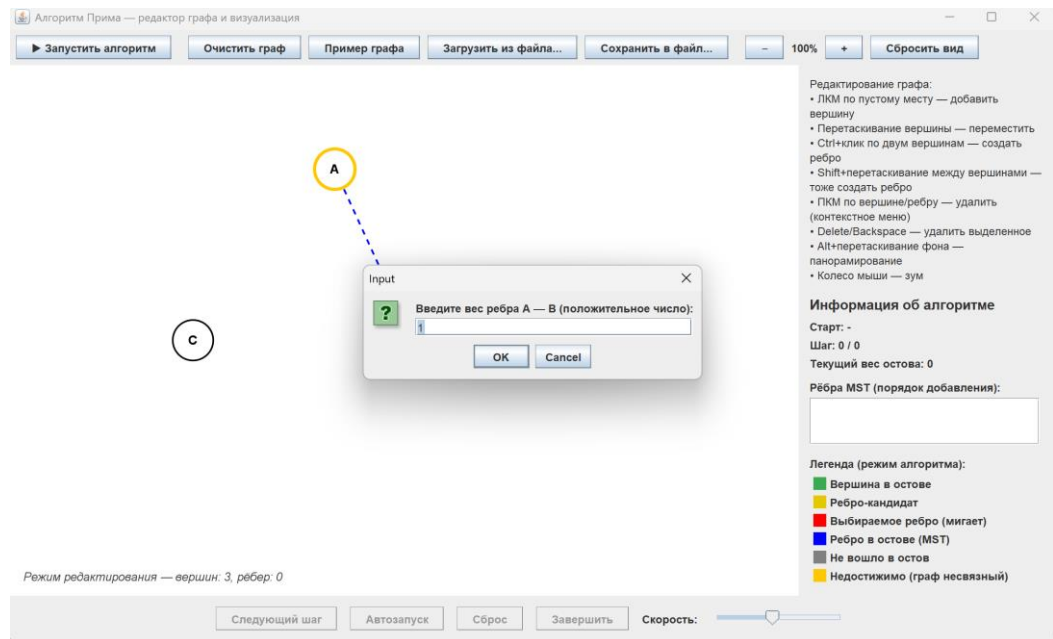


Рисунок 3 — Диалог ввода веса ребра

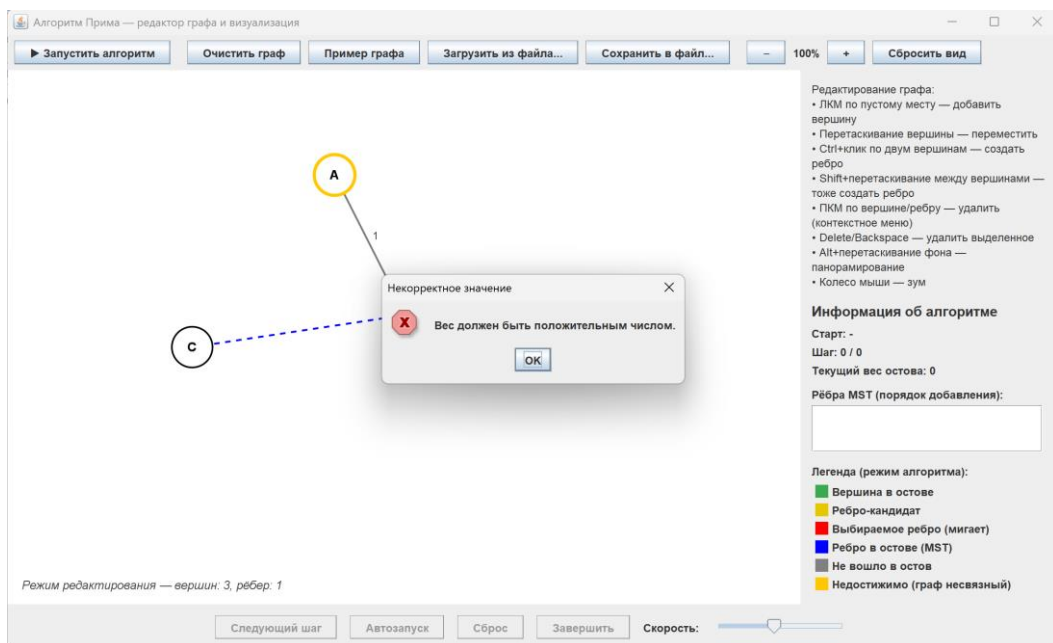


Рисунок 4 — Ошибка ввода веса ребра

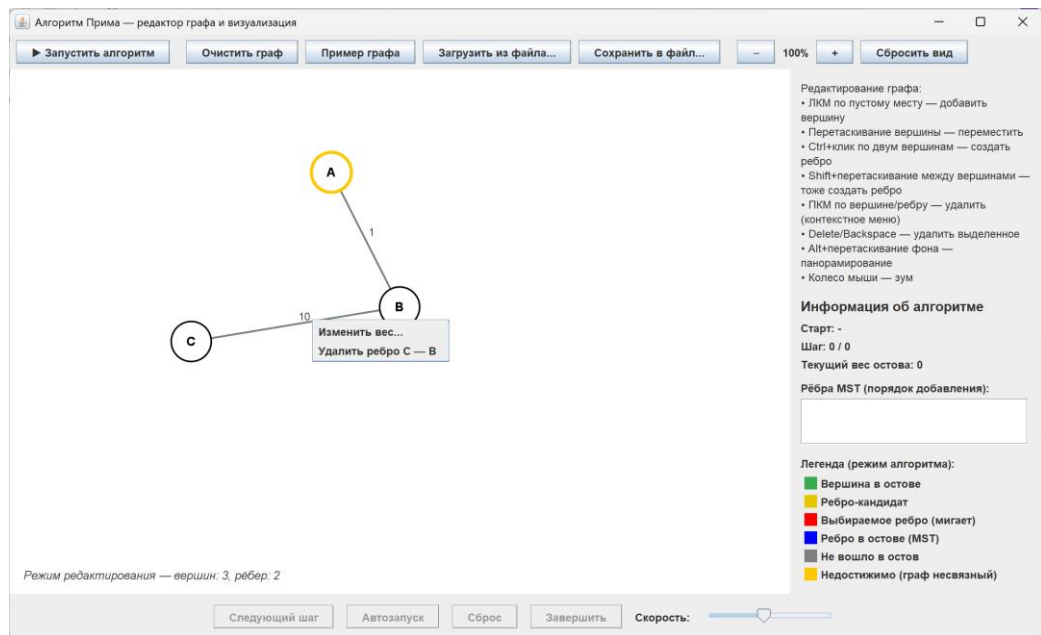


Рисунок 5 — Контекстное меню ребра с опциями изменения веса и удаления

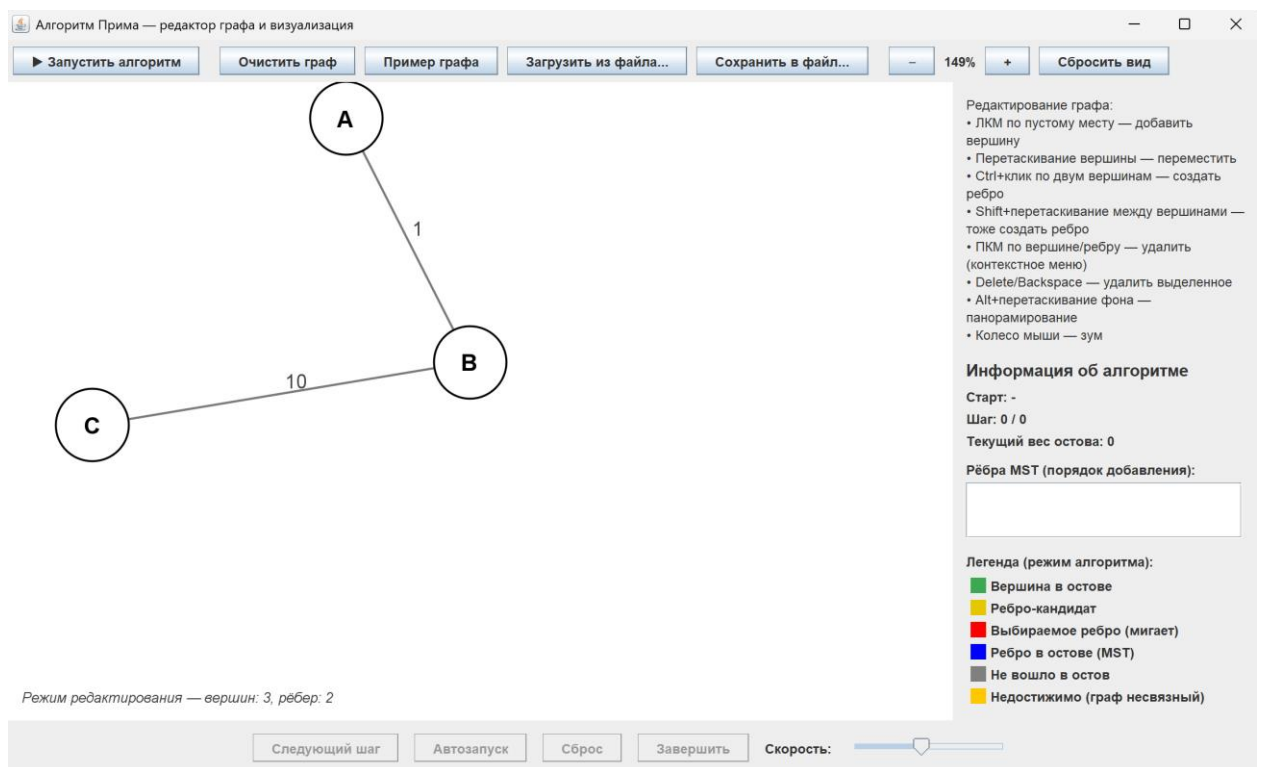


Рисунок 6 — Масштабирование холста (увеличение до 149%)

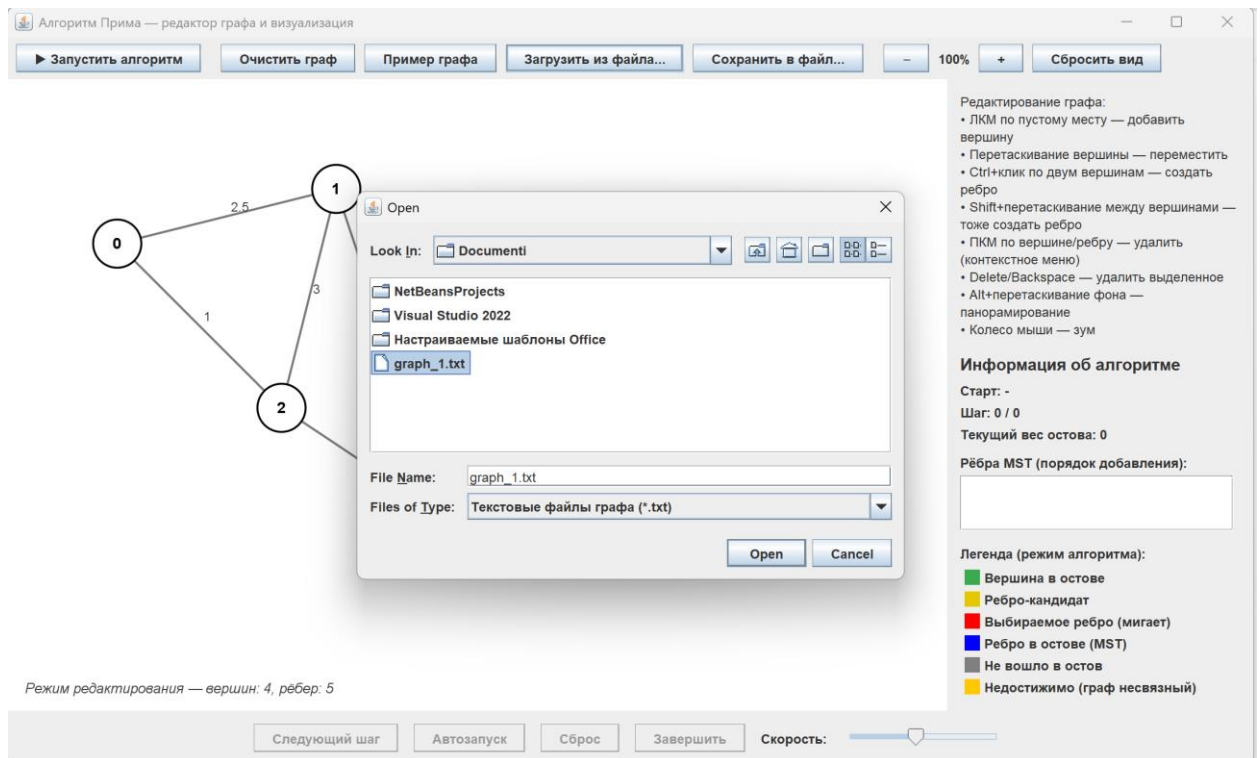


Рисунок 7 — Успешный импорт файла(если не удаётся загрузить выдаст ошибку)

Все кнопки нижней панели управления в режиме редактирования заблокированы, а при переходе в режим алгоритма становятся активными (кроме «Следующий шаг» и «Автозапуск» до инициализации). Поведение интерфейса полностью соответствует ожидаемому.

2.3.3.3. Тестирование алгоритма

Алгоритм Прима тестировался на нескольких графах, в том числе на примере из спецификации (9 вершин, 14 рёбер) и на специально подобранных крайних случаях. Для каждого теста фиксировались промежуточные и финальные состояния.

Пошаговый режим. На каждом шаге выполняется проверка:

- рёбра-кандидаты (пересекающие разрез) выделяются жёлтым цветом (рис. 8);
- выбираемое ребро с минимальным весом мигает красным (рис. 9);
- после добавления ребра новая вершина окрашивается в зелёный, а ребро — в синий (рис. 10);

- в правой панели обновляется список добавленных рёбер и общий вес.

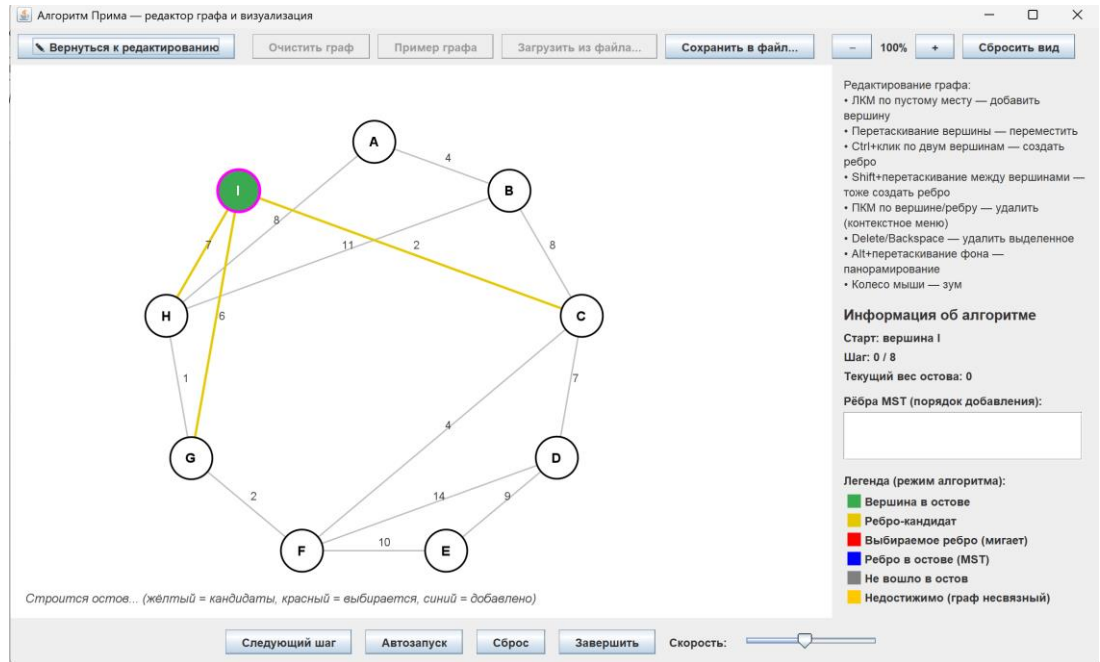


Рисунок 8 — Шаг алгоритма: кандидаты выделены жёлтым

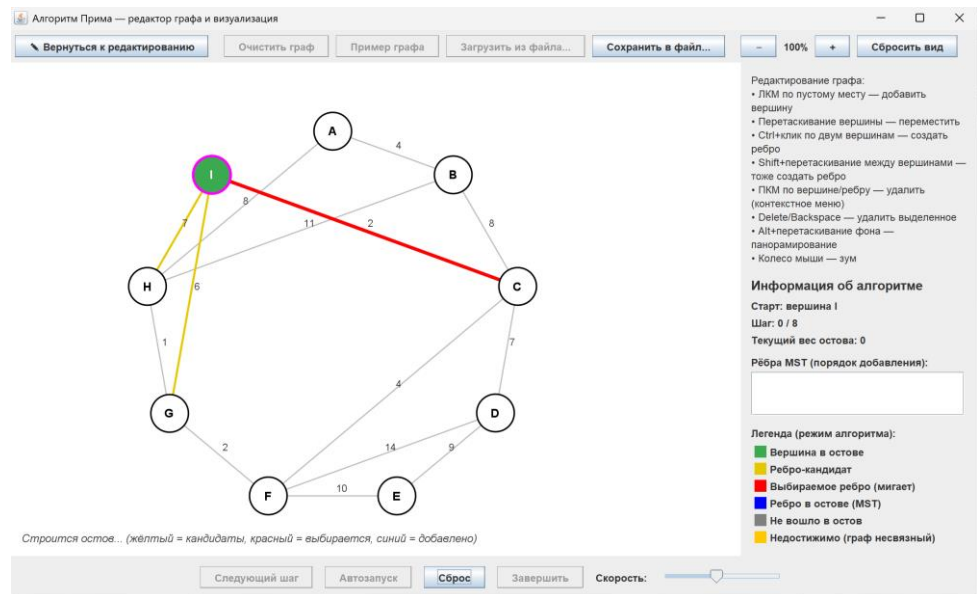


Рисунок 9 — Шаг алгоритма: минимальное ребро мигает красным

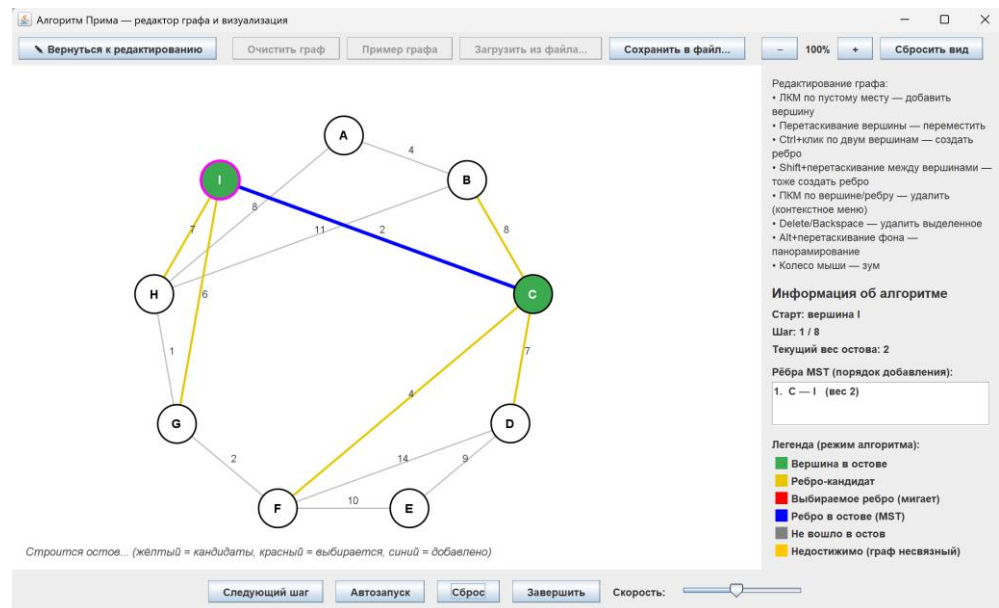


Рисунок 10 — Шаг алгоритма: ребро добавлено в остов (синее), вершина стала зелёной

Функция «Шаг назад» позволяет откатить последнее добавление и пересчитать состояние; проверено, что после отката список рёбер и вес восстанавливаются корректно.

Автоматический режим. При включении автозапуска алгоритм выполняется непрерывно с задержкой, регулируемой ползунком «Скорость» (рис. 11). Кнопка переключается на «Стоп», при нажатии выполнение приостанавливается.

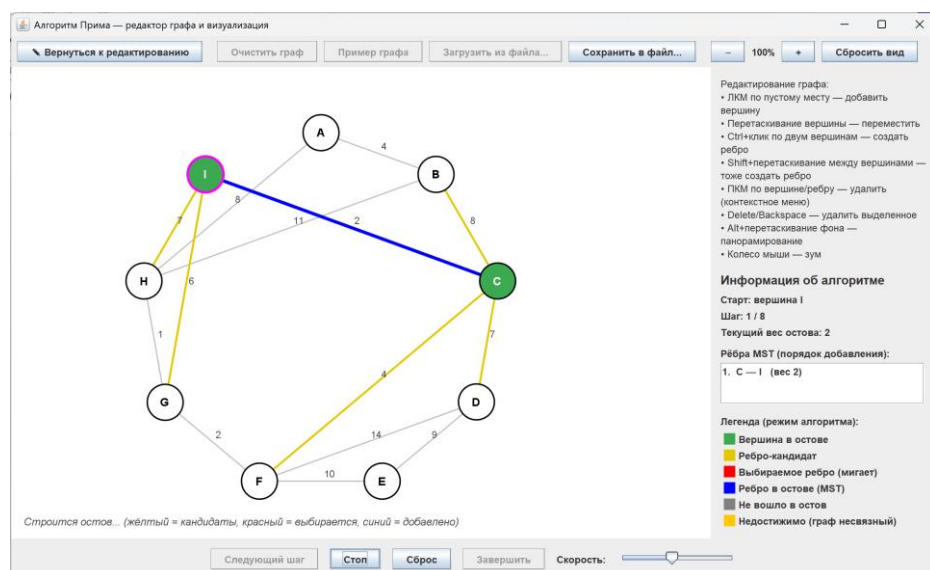


Рисунок 11 — Режим автозапуска (кнопка «Стоп», ползунок скорости в среднем положении)

Завершение и результат. По достижении всех вершин алгоритм завершается, все рёбра MST отображаются жирными синими линиями, остальные рёбра становятся полупрозрачными (рис. 12). В информационной панели отображается полный список рёбер в порядке добавления и итоговый вес. Если граф несвязный, алгоритм не запускается, а непосещённые вершины подсвечиваются оранжевым (рис. 13).

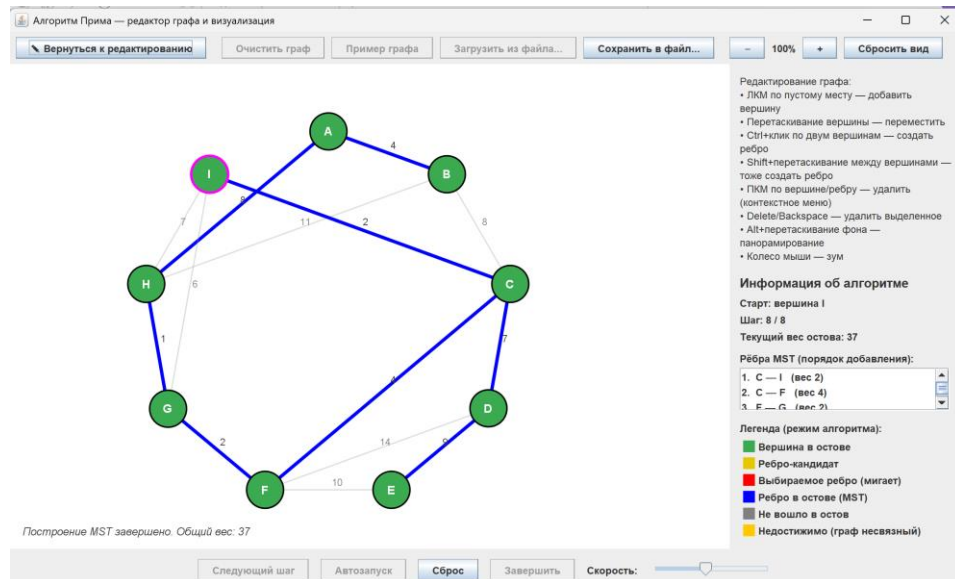


Рисунок 12 — Финальное состояние: MST построено, общий вес отображён

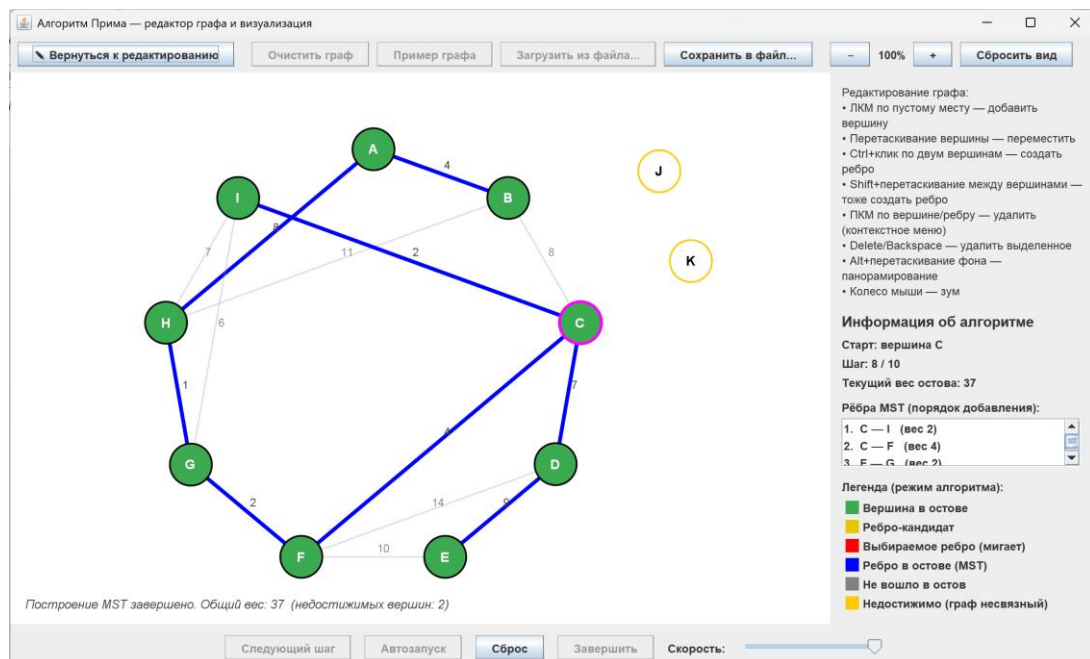


Рисунок 13 — Пример несвязного графа (оранжевые вершины недостижимы)

Сохранение результатов. После завершения (или на любом промежуточном шаге) результаты можно сохранить в текстовый файл. На рис. 14 показан диалог сохранения, а на рис. 15 – пример содержимого сохранённого файла.

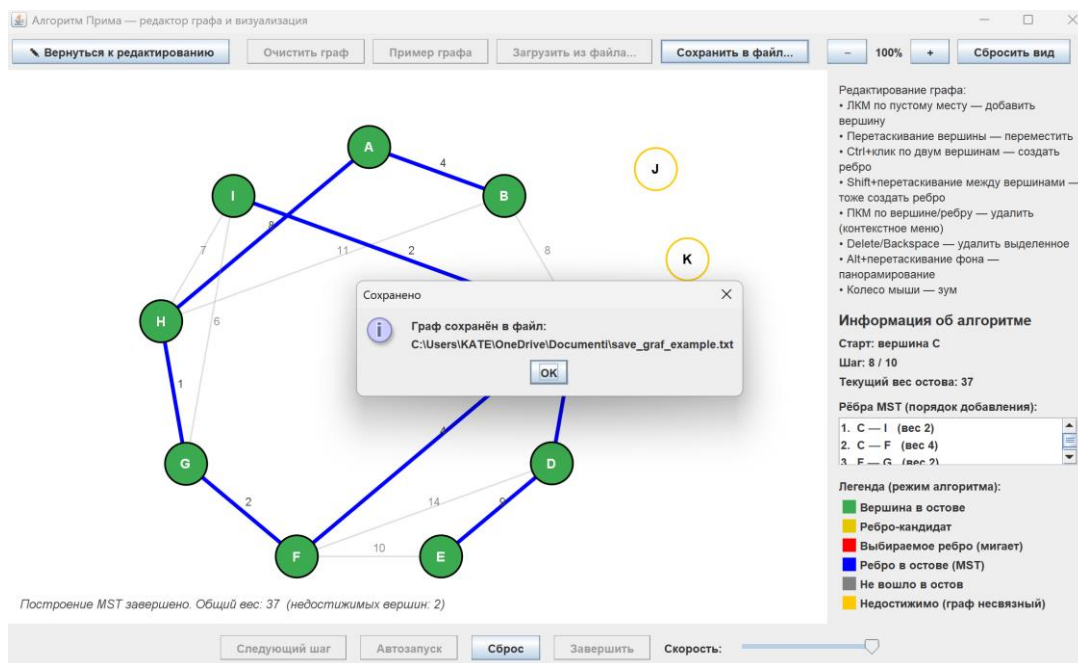


Рисунок 14 — Диалог сохранения результата

```

Результат построения минимального остовного дерева (алгоритм Прима)
Стартовая вершина: I
Вершин в графе: 9
Рёбер в осто́ве: 8

Общий вес минимального остовного дерева: 37

Список рёбер дерева в порядке их добавления:
1. C - I (вес 2)
2. C - F (вес 4)
3. F - G (вес 2)
4. G - H (вес 1)
5. C - D (вес 7)
6. A - H (вес 8)
7. A - B (вес 4)
8. D - E (вес 9)
  
```

Рисунок 15 — Содержимое файла результата (вес и список рёбер)

Граничные случаи. Проведены тесты:

- граф с одной вершиной – алгоритм завершается без добавления рёбер
- граф с двумя вершинами и ребром – ребро добавляется

- рёбра с равными весами – алгоритм выбирает первое попавшееся, вес остова остаётся минимальным
- попытка ввода отрицательного или нулевого веса – диалог отвергает значение и просит ввести положительное число

Все тесты пройдены успешно, выявленные в процессе ошибки были успешно исправлены. Программа работает стабильно, визуализация наглядна и соответствует образовательным целям.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

- **Язык программирования:** Java (версия 17). При разработке использовалась официальная документация по стандартной библиотеке Java [4].
- **Графическая библиотека:** Swing (javax.swing) – для построения интерфейса, обработки событий и рисования 2D-графики. Основные принципы работы с Swing были изучены по официальному руководству [5].
- **Стандартные библиотеки:**
 - java.awt, java.awt.geom – для графических примитивов.
 - javax.swing.Timer – для анимации и автозапуска.
 - java.io, java.nio.charset – для ввода/вывода файлов.
 - java.util (ArrayList, HashSet, PriorityQueue, Scanner, Random и др.) – для работы с коллекциями и генерации случайных данных.
- **Средства разработки:** IntelliJ IDEA, Git, GitHub.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на Отлично(рис. 15).

Отзыв

о прохождении учебной практики

Тарабанов Арсений Игоревич, студент группы 4388, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил учебную практику по теме “реализация ~~визуализатора алгоритма Прима на языке программирования Java~~” с 06.07.26 по 18.07.26

В течении практики обучающийся Тарабанов Арсений Игоревич выполнял реализацию программы на Java

В результате практики обучающийся научился языку программирования Java. Получаемую работу обучающийся Тарабанов Арсений Игоревич выполнял работу вовремя, не подводя команду.

По результатам работы Тарабанова Арсения Игоревича учебную практику оцениваю на Отлично.

Руководитель практики

Ассистент каф. МОЭВМ 15.07.26



Шестопалов Р.П.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практики была успешно разработана программа для визуализации алгоритма Прима. Все поставленные задачи решены:

1. Изучены язык Java, библиотека Swing и принципы построения графических интерфейсов.
2. Составлена детальная спецификация и план разработки, распределены роли.
3. Реализовано полнофункциональное приложение, включающее:
 - интерактивное создание и редактирование графа;
 - пошаговый и автоматический режимы выполнения алгоритма с наглядной цветовой индикацией;
 - импорт графа из файла и экспорт результата;
 - генерацию случайного графа;
 - масштабирование и панорамирование холста.
4. Проведено тестирование, все выявленные ошибки исправлены.

Разработанный визуализатор может быть использован в учебном процессе для демонстрации работы алгоритма Прима, а также как основа для дальнейшего расширения функциональности (например, добавление других алгоритмов на графах).

Вывод: В ходе выполнения учебной практики была успешно разработана программа для визуализации алгоритма Прима на языке Java с использованием библиотеки Swing. В процессе работы были изучены современные технологии разработки графических приложений, освоены принципы событийно-ориентированного программирования и работы с 2D-графикой. Разработана модульная архитектура проекта с разделением на пакеты `model`, `ui` и `main`, что обеспечивает удобство поддержки и дальнейшего

расширения. Реализован алгоритм построения минимального остовного дерева с поддержкой пошагового выполнения, автозапуска с регулируемой скоростью и возможностью отката шагов. Создан удобный интерактивный интерфейс, позволяющий пользователю создавать и редактировать граф, загружать данные из файла, наблюдать наглядную цветовую индикацию всех этапов алгоритма и сохранять результаты. Проведено тестирование на различных графах, включая граничные случаи, все выявленные ошибки исправлены. Программа полностью соответствует поставленной задаче и может быть использована в образовательных целях для наглядной демонстрации работы алгоритма Прима.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.
2. Хорстманн, К. Java. Библиотека профессионала. Том 1. Основы / К. Хорстманн, Г. Корнелл. – 10-е изд. – М.: Вильямс, 2019. – 816 с.
3. GitHub. Репозиторий проекта [Электронный ресурс]. – URL: <https://github.com/ArseniyTarabanov4388/Practice.git> (дата обращения: 18.07.2026).
4. Oracle Corporation. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. – URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 06.07.2026).
5. Oracle Corporation. Creating a GUI With Swing – A Tutorial [Электронный ресурс]. – URL: <https://docs.oracle.com/javase/tutorial/uiswing/> (дата обращения: 06.07.2026).

ПРИЛОЖЕНИЕ А

Репозиторий GitHub: <https://github.com/ArseniyTarabanov4388/Practice.git>